

for one row x_i :

$$y_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

each output is positive, and the row sums to approx. 1.

→ the direct formula is unsafe about "overflows in float32"

stable softmax first subtracts the row max:

$$m = \max_j x_j$$
$$y_i = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}$$

1. find the max
2. compute exp. and their sum
3. div. each exp. by their sum.

Ex: $[1000, 1001, 1002]$

$$= [1000 - 1002, 1001 - 1002, 1002 - 1002]$$

$$= [-2, -1, 0]$$

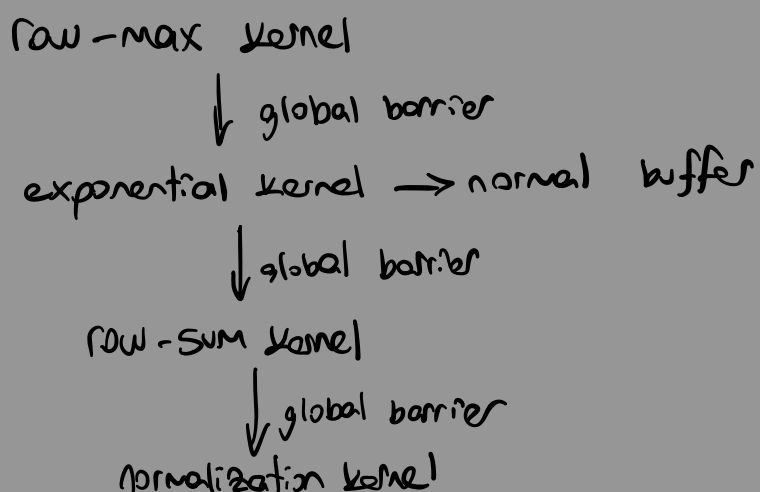
$$e^x = [0.1353, 0.3679, 1]$$

$$\text{denom} = 0.1353 + 0.3679 + 1 = 1.5032$$

$$\text{softmax} = [0.0500, 0.2447, 0.6672]$$

- Final Summary -

1. Naïve MULTI-kernel

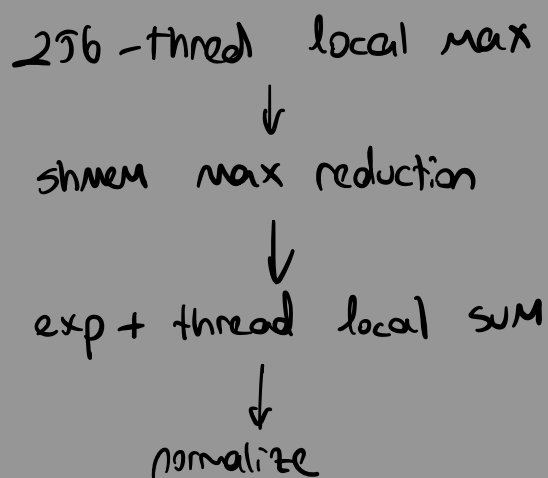


Separate launches make dependencies simple and globally synchronized.

The price is:

launch overhead,
 extra global-mem traffic,
 full temp. buffers

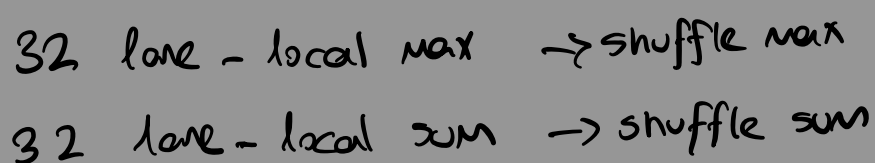
2. One-block per row



fuses the ^{whole} operation into one launch

temp. writes numerators into 'output'
 then reads and overwrites them during normalization

3. One warp per row



values move directly between lane registers
 Removing

↳ shmem and block-wide barriers

Giving good short-row performance

4. Vectorized Block

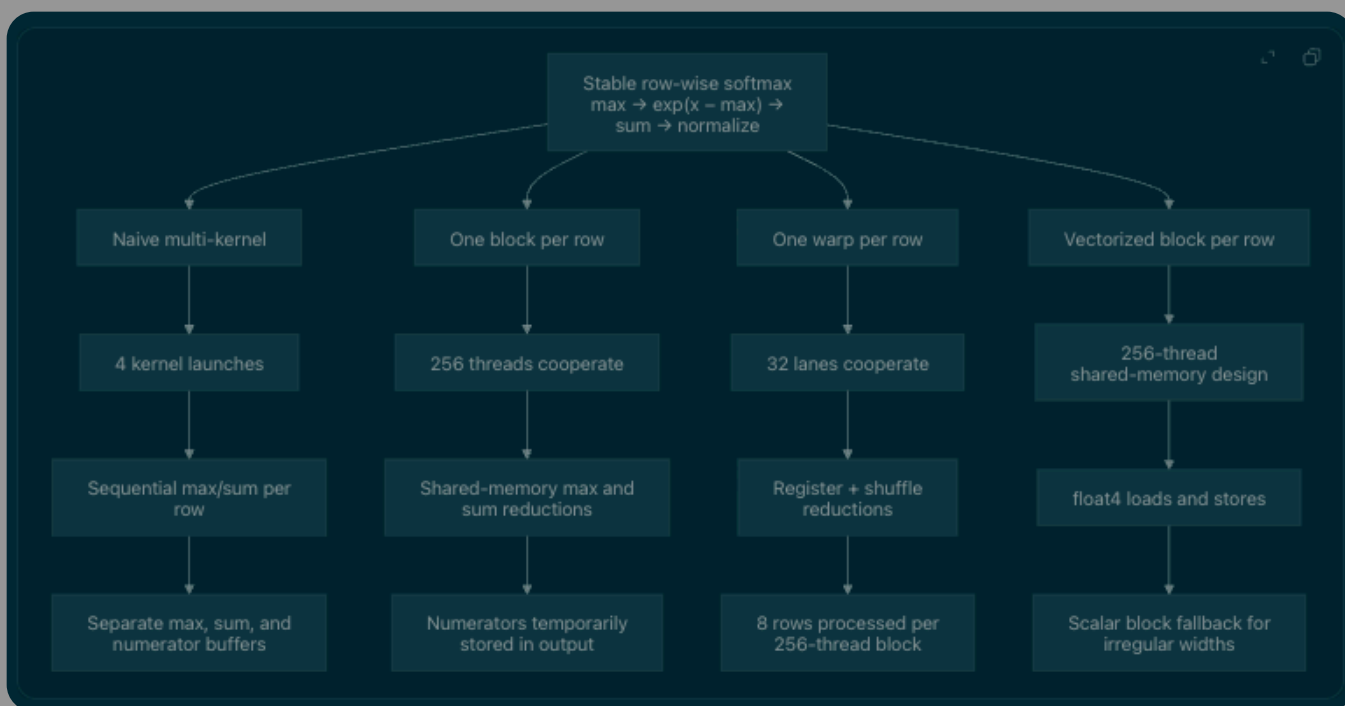
float 4 = $\{x, y, z, w\}$

↳ uses block algo.

but handles 4 adjacents per memory op.

→ no guarantee faster-ness
The math. workload
↳ 4 exp., reductions, and synchronization
still exists

→ when cols % 4 != 0
launcher falls back to scalar block softmax



"Why subtract the maximum?"

⇒ to avoid overflows in float32 (large positive logits)

"Why is naive multi-pass version slow?"

⇒ extra global memory traffic ; slow global memory
launch func. overhead
full temporary buffer

"what memory traffic is removed by fusion?"

⇒ fused the whole operation into one CPU launch
removed launch overhead
+ repeated reads and writes

? "when should one warp process a row?"

- rows are small
- 32 lanes provide enough parallelism
- avoiding shmem

? "when should one block process a row?"

- rows are wide
- shmem reductions are worth their sync. cost
- one warp would make every lane process too many cols.

? "why does performance change dramatically

with row width"

- total work and memory traffic increases
- more loop iterations
- resource, occupancy, caches and register pressure change

"which shapes favor Triton and which favor CUDA?"

→ Triton

- moderate or wide rows
- widths close to pow of 2
- cases that are concise, quickly tuned

CUDA

- short rows (warp-per-row)
- large rows (block-per-row)
- when explicit control and adjustments needed